

Federated Intrusion Detection System Based on Unsupervised Machine Learning

Maxime Gourceyraud, Rim Ben Salem, Christopher Neal, Frédéric Cuppens,
and Nora Boulahia Cuppens

Polytechnique Montréal, Montréal, QC, H3T 1J4, Canada

{maxime.gourceyraud, rim.ben-salem, christopher.neal, frederic.cuppens,
nora.boulahia-cuppens}@polymtl.ca

Abstract. Recent Intrusion Detection System (IDS) research has increasingly moved towards the adoption of machine learning methods. However, most of these systems rely on supervised learning approaches, necessitating a fully labeled training set. In the realm of network intrusion detection, the requirement for extensive labeling can become impractically burdensome. Moreover, while IDS training could benefit from inter-company knowledge sharing, the sensitive nature of cybersecurity data often precludes such cooperation. To address these challenges, we propose an IDS architecture that utilizes unsupervised learning to reduce the need for labeling. We further facilitate collaborative learning through the implementation of a federated learning framework. To enhance privacy beyond what current federated clustering models offer, we introduce an innovative federated K-means++ initialization technique. Our findings indicate that transitioning from a centralized to a federated setup does not significantly diminish performance.

Keywords: Federated Intrusion Detection · Federated K-Means++ · Federated Silhouette · Unsupervised Learning

1 Introduction

Cybercrime has become a major concern worldwide, posing significant economic problems. In fact, the World Trade Organization (WTO) estimates that the cost of cybercrime reached \$8,150 billion USD in 2023 and could escalate to \$13,820 billion USD by 2028 [21]. For organizations globally, detecting cyberattacks is crucial to mitigating their impact. Furthermore, with the increasing frequency of attacks, automating the detection process through Intrusion Detection Systems (IDS) software has become essential. Traditionally, intrusion detection involves the use of signatures, where the IDS maintains a database of known attacks and compares incoming data against this database. A match with any known attack results in the data being labeled accordingly. While effective at identifying known threats, this method relies on static rules and is inadequate for adapting to new, evolving attacks. However, with advances in Machine Learning (ML), IDS systems have been increasingly able to identify previously unseen attacks.

In recent years, ML-based IDS platforms have predominantly utilized supervised algorithms, to learn attack patterns by analyzing labeled data. However, the sheer volume of data generated by an organization’s normal operations is too vast to label entirely, making it labor-intensive and resource-intensive. To circumvent this issue, unsupervised ML algorithms, which do not require extensive data labeling, can be employed. For instance, data clustering algorithms group data into partitions, or clusters, without prior knowledge of labels. A popular algorithm for data clustering is the K-Means algorithm, introduced by S. Lloyd [9], which creates clusters based on the proximity of data points to cluster centroids.

ML thrives on large, diverse datasets, which can pose challenges for organizations not primarily focused on data tasks. Collaboratively developing an IDS using varied data can leverage shared insights. However, traditional ML frameworks are centralized, requiring data to be consolidated at a single location. This centralization poses privacy risks and creates a single point of failure. For example, a compromised server holding data from all participants exposes extensive sensitive information. Conversely, decentralized data storage across departments or even different organizations force an attacker to breach each one individually, significantly reducing overall risk. The field of Federated Learning (FL), introduced by McMahan et al. [11], addresses these issues by proposing a learning protocol that includes model learning at the client level and model aggregation at the server side, thus keeping the data localized.

This paper combines unsupervised and federated learning to develop an IDS. We validate this approach with experiments using the UNSW-NB15 [13] and CIC-IDS2017 [18] datasets. We summarize the contributions of this work as follows:

- We propose a federated learning-compatible architecture that integrates clustering algorithms for intrusion detection.
- We introduce a federated k-means++ initialization that is mathematically equivalent to its centralized counterpart.
- We present a method for computing a simplified silhouette score for clustering model selection within a federated framework.

The remainder of the paper is outlined as follows. A review of related work is provided in Section 2. The methodology of this work is detailed in Section 3. The experimental results of our approach are reported in Section 4. We discuss our findings in Section 5 and conclude this work in Section 6.

2 Related Work

This section presents an overview of clustering algorithms that are applicable in FL environments, followed by an introduction to relevant centralized and federated IDS frameworks.

2.1 Clustering Algorithms

Client Clustering. One part of the federated clustering is not dedicated to cluster the clients' data but the clients themselves. This way, the clients are grouped in the aggregation step to enhance the performances of a supervised model. For example, CADIS [14] uses the second last layer of locally learned deep learning models to calculate a customer similarity matrix. Using a layer of the model learned at the customer's site makes it possible to observe the distribution of data without having access to the data itself. When the models are aggregated by the server, the modifications made locally are weighted by the size of the client's dataset and the inverse of the number of clients that are similar to it. This way, the weight of clients is reduced for those belonging to a large cluster and increased for those belonging to a small cluster.

Federated Clustering. The type of federated clustering algorithms that we are interested in our study are those that cluster the clients' data. One of the first introduced was K-FED [4] and it is based on K-Means. The clients perform the clustering algorithm of Aswathi et al. [3]. The server aggregates the centroids by using Lloyd's algorithm [9]. K-FED works in only one round of communication. Each client must not have more than $\sqrt{k}$ of the overall k clusters.

Two federated clustering algorithms are presented approximately at the same time. First, the Federated K-Means by Garst and Reinders [5] uses a K-Means++ initialization [8] at the client level. Then, the server aggregates the centroids by performing Lloyd's algorithm with each centroid weighted by the size of the cluster it represents. The clients improve the model by doing one step of Lloyd's algorithm on their data with the global centroids. A new round of communication begins after the server once again aggregates the clients' centroids. The algorithm stops after a fixed number of rounds of communication.

The other algorithm presented around the same period of time as [5] is proposed by Holzer et al. [7]. This algorithm does not require that every client participates in the protocol at the same time. Each selected client updates the global centroids with one step of Lloyd's algorithm. The server aggregates the new centroids with a weighted average cluster by cluster. The weight of each centroid is the size of its cluster on the client side. This approach is similar to the one that is used in Tensorflow Federated [20]. But Holzer et al. use a learning rate to bound the updates and use inertia to limit the fluctuations of the global model.

2.2 Unsupervised IDS

In this section, we highlight related IDS work based on unsupervised learning that utilizes either a centralized or federated learning approach, then provide the summary of their results in Table 1. We focus on works that utilize the same datasets as in this paper (i.e. UNSW-NB15 and CIC-IDS2017), as we feel this offers a fair comparison.

Centralized IDS. Prasad et al. [16] propose a clustering algorithm that creates arbitrary shapes, not only convex shapes. To achieve this, their method initializes a large number of initial clusters. These clusters are grouped by similarity. They accompany their method with a selection of variables. They achieve respective accuracies of 0.8860 and 0.8828 on the UNSW-NB15 and CIC-IDS2017 datasets.

For their anomaly detection, Yang et al. [22] train an auto-encoder on normal data. In addition to detecting anomalous data with a threshold, the proposed method quantifies the normal distribution (i.e. on normal data) of hidden layers with their mean and variance. During the detection phase, the value of each of the hidden layers is compared with normality using the Mahalanobis distance [10]. The final anomaly score is a linear combination of the reconstruction error and all the Mahalanobis distances of the hidden layers. Then, they determine a threshold for this score to differentiate the attacks from the normal traffic. With their method, the authors obtained an accuracy of 0.85 ± 0.01 and an F_1 score of 0.84 ± 0.01 .

Federated IDS. Md Tayeen et al. [12] propose a federated learning algorithm using FedAvg [11]. They train an auto-encoder in a federated manner on normal data and then perform anomaly detection. However, the only parameters sent to clients are the weights of the layer preceding the latent space and those of the layer following it. Strictly speaking, federated training only concerns the latent space. The F_1 score obtained on UNSW-NB15 is 0.91 ± 0.003 .

Grammenos et al. [6] introduce a federated streaming and differentially private algorithm for computing Principal Component Analysis (PCA). Federated-PCA estimates the rank of PCA with unknown distribution in a memory-limited setting. While the reported numerical simulations highlight the algorithm’s potential, its performance when faced with outliers or missing data remains to be evaluated. The work of Nguyen et al. [15] is in the same vein as [6] and uses Federated PCA on Grassmann Manifold for Internet of Things (IoT) anomaly detection. Their unsupervised framework for efficient host-based IDS is formulated as a consensus optimization problem with the aim of achieving privacy-preservation and communication-efficiency.

Aouedi et al. [1] propose a supervised IDS enhanced with unsupervised learning. They train an auto-encoder in a federated way. Afterwards, they train on the server side a neural network with little labeled data. This is not exactly an unsupervised model but we find this work as important to highlight as the intrusion detection literature in this domain is sparse.

Reference	Dataset	Setup	Accuracy	F ₁
Prasaad et al. [16]	CIC-IDS2017	Centralized	0.8860	0.8828
Sirisha et al. [19] (k-means)	CIC-IDS2017	Centralized	0.79	0.88
Sirisha et al. [19] (IForests)	CIC-IDS2017	Centralized	0.37	0.91
Yang et al. [22]	UNSW-NB15	Centralized	0.85	0.84
Md Tayeen et al. [12]	UNSW-NB15	Federated	/	0.91
Aouedi et al. [1]	UNSW-NB15	Federated	0.8432	/

Table 1: Summary of the performances of relevant work

3 Methodology

3.1 IDS architecture

The proposed IDS architecture is represented in Figure 1. It is composed of three main components: clustering, expertise, and classification. Data used by the IDS is first regrouped in clusters. Then, these clusters are analysed by the clients of the federated setup. Using the aforementioned expertise, each cluster is given a label according to its composition. Data is then classified as the label of the cluster it belongs to. The rest of this section describes each component.

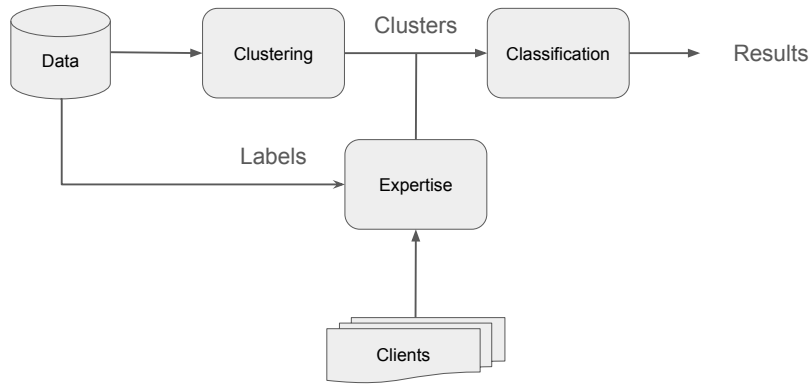


Fig. 1: Representation of the proposed IDS

3.2 Federated Clustering

The clustering part of the IDS, as shown in Figure 1 is either centralized or federated. To perform the clustering in a centralized manner, we use K-Means

clustering [9]. In the federated setup we used Federated K-Means clustering as proposed by Garst and Reinders [5]. The behavior of the IDS in a centralized setup is used as a baseline for the performance of the federated setup.

However, the initialization of the Federated K-Means clustering of Garst and Reinders [5] requires that every client participating in the protocol sends to the server the results of the k-means++ initialization [2] on their data. Let $k \in \mathbb{N}^*$ be the number of clusters required for the clustering and $N \in \mathbb{N}^*$ be the number of clients. The protocol introduced by Garst and Reinders requires that the clients disclose a total of kN entries. In this section, we propose the federated k-means++ initialization to enhance the privacy of the clustering algorithm.

K-Means Properties. Before introducing the Federated K-means++ initialization, we need to reformulate the probability distributions used in K-Means++ [2]. Let $(X_j)_{j \in \{1, \dots, N\}}$ be the datasets of the clients $(\mathcal{C}_j)_{j \in \{1, \dots, N\}}$ and $X = \bigcup_{j=1}^N X_j$ the total dataset. Suppose that each $x \in X$ is either unique or there is a way to distinguish all the entries with the same values. For each $x \in X$, we have:

$$P_{\mathcal{U}}(x) = \frac{1}{|X|} = \frac{|X_j|}{|X|} \times \frac{1}{|X_j|} = P_{\mathcal{U}}(C_j)P_{\mathcal{U}}(x|C_j) \quad (1)$$

Equation 1 shows that the uniform distribution on a dataset X can be equivalently expressed as a choice of a client C_j that then realizes the uniform sampling on its own data. Similarly, the D^2 distribution introduced for the k-means++ initialization [2] can be expressed as choice of a client and this client samples its data with a local D^2 distribution. This is shown by equation 2. Let c be the set of centroids at any moment. Let for all j , $Z_j = \sum_{x \in X_j} d(x, c)$ with $d(x, c) = \min_{i \in \{1, \dots, |c|\}} (\|x - c_i\|_2^2)$ and $Z = \sum_{j=1}^N Z_j = \sum_{x \in X} d(x, c)$. The D^2 probability distribution can be written as:

$$P_{\mathcal{D}^2}(x) = \frac{d(x, c)}{Z} = P(C_j)P_{\mathcal{D}^2}(x|C_j) = \frac{Z_j}{Z} \times \frac{d(x, c)}{Z_j} \quad (2)$$

So the probability distributions used in the k-means++ initialisation can be written in a decentralized way for any distribution of data between the clients. Also the formulation of the probability distributions does not allow the server to access the clients' data.

Federated K-Means++ Initialization. Algorithm 1 uses the properties discussed above to create a federated version of the K-Means++ algorithm equivalent to its centralized counterpart. With this algorithm, the server only accesses the centroids selected by the clients, and does not access to the rest of their data.

With Algorithm 1, the clients only disclose the sampled data to the rest of the participants. If k points are required, then the clients only reveal k points and this initialization is equivalent to the centralized one as shown previously.

Algorithm 1 Federated K-Means++ Initialization

Require: $k, \mathcal{C} = (\mathcal{C}_j)_j, (X_j)_j$

- 1: The clients send the size of their datasets to the server.
- 2: The server samples a client $\mathcal{C}_j$ in $\mathcal{C}$ with $P(\mathcal{C}_j) = \frac{|X_j|}{|X|}$
- 3: The client $\mathcal{C}_j$ samples the first centroid c_1 uniformly on X_j
- 4: $c \leftarrow \{c_1\}$
- 5: **for** $i \in \{2, \dots, k\}$ **do**
- 6: The server samples a client $\mathcal{C}_{i'}$ in $\mathcal{C}$ with $\forall j, P(\mathcal{C}_j) = \frac{Z_j}{Z}$
- 7: The client $\mathcal{C}_{i'}$ samples c_i where $P(c_i|\mathcal{C}_{i'}) = \frac{d(c_i, c)}{Z_{i'}}$
- 8: $c \leftarrow c \cup \{c_i\}$
- return** c

Algorithm 2 explains how we insert the federated k-means++ initialization into Garst and Reinders' federated K-Means approach. c^j are the centroids of the j -th client and C^j are the clusters associated with them. We note s^j the size of the clusters of the j -th client and c^g the global centroids.

3.3 Federated Simplified Silhouette

K-Means clustering has two main issues: the initialization and the choice of the number of clusters k . We cover the issue of initialization in Section 3.2. Since clustering is used an unsupervised set of techniques, metrics such as accuracies are other performance metrics cannot be used. Now, we introduce the federated simplified silhouette score. The silhouette score [17] is a metric to estimate how good a clustering is for each data point. To reduce the computation cost, Hruschka et al. [8] proposed a simplified silhouette. We use the latter because it is less costly and because the classical silhouette score would require each client to know where each data point of the others are. This would undermine the efforts to improve privacy. For each $x \in X$, let's note $c(x)$ the centroid associated with x . Let c be the set of the centroids and d be a distance. The simplified silhouette score is computed as follows:

$$\begin{cases} a(x) = d(x, c(x)) \\ b(x) = \min_{i \in \{1, \dots, k\}, c_i \neq c(x)} d(x, c_i) \\ s(x) = \frac{b(x) - a(x)}{\max(a(x), b(x))} \end{cases}$$

We adapt the formula of s to a federated setup. For each client $\mathcal{C}_j$, we have the average silhouette score of $\mathcal{C}_j$:

$$\overline{S_{C_j}} = \frac{1}{|X_j|} \sum_{x \in X_j} s(x) \quad (3)$$

Then the server computes the average score weighted by the size of each dataset across the network :

Algorithm 2 Federated K-Means with Federated K-Means++ initialization**Require:** k

Federated k-means++ initialization

 $c_g \leftarrow \text{federated_kmeans++}(k, \mathcal{C}, (X_j)_j,)$ **for** round r **do** **For each client** $\mathcal{C}_j \in \mathcal{C}$: $c^j \leftarrow c^g$ Determine s^j # The client $\mathcal{C}_j$ ignores the centroids without data associated with. $s^j \leftarrow C^j[s! = 0 \text{ for } s \text{ in } s^j]$ $k_j \leftarrow \text{size}(s_j)$

The clients compute one step of Lloyd's algorithm

 # Lloyd's algorithm is applied with c^j as initial centroids. $s^j, c^j \leftarrow \text{kmeans}(X_j, k_j, \text{init} = c^j)$ Send s^j, c^j to the server. **At the server :**

Centroids and size of clusters are concatenated.

It creates a dataset on which the server applies k-means.

 $c \leftarrow [c^1 | c^2 | \dots | c^N]$ $s \leftarrow [s^1 | s^2 | \dots | s^N]$

Aggregation step. Lloyd's algorithm weighted by the clusters' size.

 $c^g \leftarrow \text{kmeans}(c, k, \text{weights} = s)$ Send c^g to the clients.

$$\begin{aligned}
S &= \frac{1}{\sum_{j=1}^N |X_j|} \sum_{j=1}^N |X_j| \overline{S_{C_j}} \\
&= \frac{1}{\sum_{j=1}^N |X_j|} \sum_{j=1}^N \sum_{x \in X_j} s(x) \\
S &= \frac{1}{|X|} \sum_{x \in X} s(x)
\end{aligned} \tag{4}$$

Therefore, the average silhouette score computed is equivalent to if the server had direct access to all the data without actually accessing it. This is the average federated simplified silhouette score. The goal is to find the best value of k by maximizing this score just like in a centralized setup. Throughout this paper, we use federated silhouette and federated simplified silhouette interchangeably.

3.4 Binary Classification

Once the clusters are formed, the IDS needs the clients to estimate the proportion of benign communications in each cluster. The server stores the proportion of benign communications in cluster i of client j in $p_{i,j}$. The clients also send to the server the size of the cluster in their dataset. Similarly, the size of cluster i of

the client j is stored in $s_{i,j}$. The server then computes the proportion of benign communications across the network for each i :

$$P_i = \frac{\sum_j p_{i,j} \times s_{i,j}}{\sum_j s_{i,j}} \quad (5)$$

In a centralized setup, the data owner directly estimates P . Once P is estimated, the rule we used is to consider the cluster i benign if $P_i > 0.5$. Contrary-wise, this cluster is considered as an attack cluster if $P_i \leq 0.5$. Once a cluster is classified as benign or attack, all the data associated with it are classified as such.

To measure the performances of the models, we set the positive class to be the attacks because it is what we want to detect. We use the classical metrics of accuracy, precision, recall and F_1 score.

3.5 Datasets

To evaluate the proposed architecture of the IDS and the federated algorithm proposed, we used two common datasets in intrusion detection: UNSW-NB15 and CIC-IDS2017. They are centralized datasets so we have to split them to create federated datasets, as explained in the following.

UNSW-NB15. The UNSW-NB15 dataset was built at the University of New South Wales to solve issues with previous datasets [13]. This dataset was designed to include realistic and various normal behavior and modern attacks. It is widely used in the field of intrusion detection to evaluate IDS. The dataset contains attacks that are relatively easy to detect such as DDoS, along with more complex attacks like exploits.

To create a federated dataset from UNSW-NB15 we choose to regroup data with respect to the value of a certain variable. For this dataset, each client is composed by only a single class. For instance, one of the clients is entirely composed of the normal communications, one is entirely composed of DDoS attacks, etc. We recognize this does not represent an ideal setup for independently distributed samples, however we see this as an acceptable compromise to create a federated dataset using UNSW-NB15.

CIC-IDS2017. The CIC-IDS2017 [18] dataset was created by the University of New Brunswick. The creators of this dataset insisted on simulating a realistic background traffic along with modern attacks. They use a diversity of protocols, devices, and operating systems to be as comprehensive and realistic as possible. Since the network was fully controlled by the researchers, each communication was labeled. The normal traffic is called "benign" and the attacks are labeled according to each type contained in the dataset. For instance, there are DDoS, Web attacks, etc.

For CIC-IDS2017, we create the federated dataset by regrouping client by destination IP of the communication. This way, each client is considered as a receiver of the communications and is closer to a realistic setup.

Data Preprocessing. Before data is used in our approach we perform the following preprocessing steps. We remove the dataset entries containing empty variables. We replace infinity values with minimum and maximum values of the corresponding variable. We replace the categorical variables by binary variables. Then, we normalize each variable so that they are limited to the $[0; 1]$ interval. We apply the unsupervised variable selection based on value frequency [16]. Redundant data is removed from the dataset. Finally, we create a training set representing 80% of the whole dataset and a test set representing 20%. The entire dataset is shuffled before creating the previously mentioned datasets.

Overview of Experimental Approach Here we outline the process used to train our federated IDS based on unsupervised learning: The algorithm is trained with values of the number of clusters k ranging from 1 to 300 for CIC-IDS2017 and from 1 to 70 for UNSW-NB15. We vary the number of rounds of communications r using the values 0, 5 and 10. Then, the chosen value of r is the minimum such that the silhouette curve has a maximum or local maximums. Following this, we choose the number of clusters such that it is the maximum, or a local maximum, of the simplified silhouette curve. The clients vote for the composition of each cluster, as shown previously. Based on this, the server aggregates the results. The class (attack/benign) of the cluster is determined according to the vote system chosen. Finally, each communication is labeled in accordance with the cluster it belongs to.

4 Results

4.1 Model selection

Figure 2 presents the evolution of the silhouette score as a function of the number of clusters. Both UNSW-NB15 and CIC-IDS2017 are represented on these figures. The blue curves represent the silhouette score computed on the clusters generated with the centralized k-means algorithm. The other curves represent the evolution of the silhouette score as a function of r , the number of rounds of communication between the clients and the server.

UNSW-NB15 For UNSW-NB15 (Figure 2a), the centralized k-means curve and the curves for $r = 5$ and $r = 10$ share the same shape. Contrarywise, the curve $r = 0$ seems flat. We tested the centralized version of k-means++ on the same data and it generated a curve similar to the one with $r = 0$. So the "flat" shape is not an error of the federated k-means++. Even if the first three curves mentioned share the same shape, the maximums are not located at the same

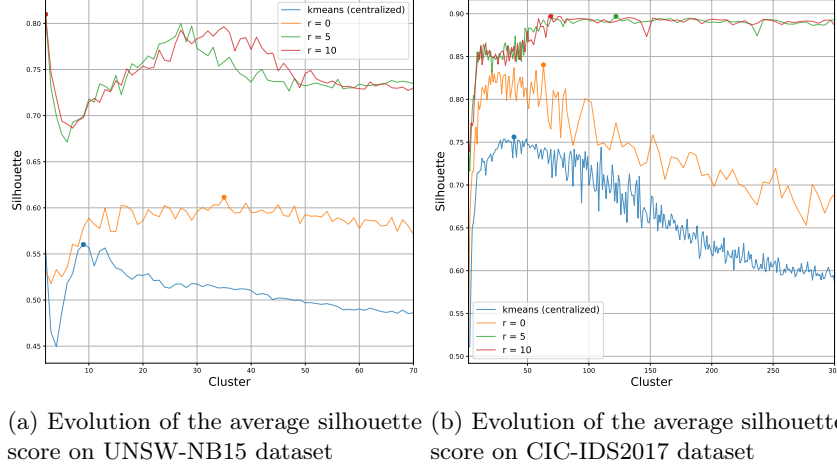


Fig. 2: Evolution of the average silhouette score as a function of the number of clusters (fed. K-Means with fed. K-Means++ init.).

value of k , the number of clusters. In fact, the curves for $r = 5$ and $r = 10$ have a maximum around $k = 30$ but the centralized k-means has a maximum at $k = 9$. So the federated models used here shifts the maximum of the silhouette score to higher values of k .

The curve with $r = 0$ shows no clear maximum. So we chose the number of round of communications tested that produces a curve with a clear maximum. In that case, we chose $r = 5$. We then select the maximum of the curve which is $k = 27$.

CIC-IDS2017 For CIC-IDS2017 (subfigure 2b), we can see that the silhouette curves for the centralized k-means and the federated k-means++ ($r = 0$) share the same shape. Contrary to UNSW-NB15, the curves for $r = 5$ and $r = 10$ do not look like the centralized one. These two last curves do not have a clear maximum.

The only federated curve whose shape resembles that of the centralized curve is with $r = 0$. So we chose this number of round of communications and the maximum of this curve which is $k = 63$.

Summary of Model Selection. We can see on figure 2 that when r increases, the silhouette score increases. It could show an improvement in the quality of clustering as the silhouette score measures this. However, the shapes of the curves in figure 2b for $r > 0$ are clearly not the kind that is expected. Indeed, a silhouette curve is expected to have at least one local maximum and to decrease as k increases. Here it is clearly not the case even for $r = 0$ and in the centralized setup, the curves have a maximum before $k = 300$. This shows that the federated

iterations do not behave as expected. The suboptimality is also present on the UNSW-NB15 dataset, the maximums of the curves $r > 0$ are shifted to higher values of k compared to the centralized setup.

So we understand that the federated models studied generate suboptimal results with respect to the silhouette score. We know that the initialization we proposed is equivalent to its centralized counterpart. But we have no *a priori* guarantee that the aggregation function from Garst and Reinders [5] that we used is also equivalent to the step of Lloyd’s algorithm. This can explain why the results are not as satisfactory as the centralized ones.

4.2 Evolution of Performance Metrics

Figure 3 presents the evolution of the F_1 score as a function of the number of clusters. Similar to figure 2, this figure shows the evolution of the F_1 score for different values of r for the federated K-Means with federated K-Means++ initialization.

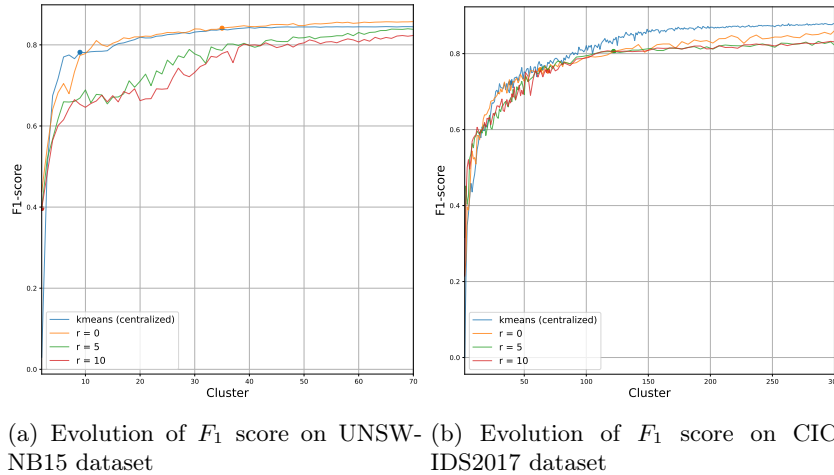


Fig. 3: Evolution of the average F_1 score as a function of the number of clusters (fed. K-Means with fed. K-Means++ init.).

UNSW-NB15 For UNSW-NB15 (figure 3a), we can see that all curves are increasing in trend. There are two groups of curves. The first one is composed of the centralized algorithm and the federated one with $k = 0$. This group shows better performances. The other one is composed of the curves with r set to 5 and 10. Even though they get closer to the other two as k increases, they are almost always below them.

CIC-IDS2017 For CIC-IDS2017 (figure 3b), all curves are increasing in trend just like for UNSW-NB15. Contrary to the latter, the federated curves on the former are closer and the centralized one is clearly above the others starting from $k = 50$. The curve $r = 0$ is above the other two federated ones between $k = 20$ and $k = 70$ and then after $k = 160$.

Summary of Performance Evolution. For both datasets, the performances of the centralized k-means are often better than the federated ones for fixed k . It means that the more the clusters, the better the performance. But more clusters also mean more expertise from the clients. If this expertise is provided by cybersecurity analysts, it can be tedious to label too many clusters. Moreover, we used unsupervised learning to limit the labeling. So the participants should be aware of such behavior before using the proposed IDS.

We also notice that the performances for $r = 0$ are equivalent or better than the other two federated scenario. In terms of performances, it is better to keep r as low as possible. So iterating on the centroids from the federated k-means++ initialization is not beneficial in term of performances. This is in line with the suboptimality of the aggregation function that we pointed in section 4.1

4.3 Performance Comparison

Table 2 and 3 compare the performances of the K-Means algorithm, federated K-Means [5] and the federated K-Means with the proposed federated K-Means++ initialization. The results are computed using UNSW-NB15 and CIC-IDS2017 respectively.

UNSW-NB15 dataset. As we can see in Table 2, the federated models have better accuracies and precisions than the centralized model. It means they produce less false positives. But both of them present a lower recall, which means that they produce more false negatives. The performances of the federated models should be compared with the number of clusters these methods need. Indeed, both of the federated algorithms require significantly more clusters than the centralized one.

Model	Accuracy	Precision	Recall	F_1	k	r
k-means (centralized)	0.7659	0.6640	0.9500	0.7817	9	/
Garst&Reinders	0.7706	0.6904	0.9201	0.7889	15	0
Fed. k-means & fed. k-means++	0.7743	0.7121	0.8456	0.7731	27	5

Table 2: Performance comparison between the models with selected hyperparameters (UNSW-NB15)

CIC-IDS2017 dataset In Table 3, we can see that the federated models have comparable or better accuracies, recalls and F_1 scores than the classical, central-

ized one. However, they have lower precisions and they need more clusters. We also notice that the selected r is 0 for both federated algorithms. As discussed in sections 4.1 and 4.2, it seems that it is not always relevant to chose $r > 0$. For our study, $r = 0$ with the federated k-means of Garst and Reinders means that there is only one aggregation step after the local k-means++ initializations.

Model	Accuracy	Precision	Recall	F_1	k	r
k-means (centralized)	0.8958	0.7505	0.6969	0.7227	39	/
Garst&Reinders	0.9121	0.7041	0.9554	0.8107	83	0
Fed. k-means & fed. k-means++	0.8947	0.6765	0.8613	0.7578	63	0

Table 3: Performance comparison between the models with selected hyperparameters (CIC-IDS2017)

5 Discussion

In Section 4.1 and 4.2, we notice that there is a suboptimality in the aggregation process. Indeed, the behavior of the model gets worse as the number of rounds increases. However, the performances of the studied models are better in terms of accuracy than the clustering algorithms from Table 1.

Even though the rounds of communication do not increase the performances of the model, the performances obtained in a centralized setup or in a federated one are similar or better. The federated models require more clusters to achieve the same performances. So the federated clustering models studied are less efficient but the performances temper the suboptimality comment. It leaves room for future research.

Since the model detects attacks, it shows that the clients did collaborate during the training steps. For example in UNSW-NB15, one of the clients has only normal communications. But with the centroids learned across the network it is now capable of detecting attacks even though they were not previously seen. Similarly, with the federated dataset we created for CIC-IDS2017, 203 datasets on 212 are only composed of normal communications. The learning process is still capable of finding attacks.

6 Conclusion

In this paper we propose an IDS architecture that uses clustering algorithms, centralized or not. We improved the privacy of the clients' data compared to the federated k-means from Garst and Reinder with our federated k-means++ initialization without any loss of performance overall. Moreover, we found that the IDS and the learning process we described enable collaboration among the clients without sharing the datasets to the server. To select the federated clustering models, we proposed an average federated simplified silhouette score. We showed it is the same as the centralized one.

Future research should focus on refining aggregation techniques to enhance the performance of federated models, potentially leading to more robust and scalable cybersecurity solutions in decentralized networks. This direction not only promises improvements in IDS efficacy but also broadens the potential for cross-organizational collaboration without compromising sensitive information.

Bibliography

- [1] Aouedi, O., Piamrat, K., Muller, G., Singh, K.: Intrusion detection for softwarized networks with semi-supervised federated learning. In: ICC 2022 - IEEE International Conference on Communications. pp. 5244–5249 (2022). <https://doi.org/10.1109/ICC45855.2022.9839042>
- [2] Arthur, D., Vassilvitskii, S.: k-means++: the advantages of careful seeding. In: Proceedings of the eighteenth annual ACM-SIAM symposium on Discrete algorithms, pp. 1027–1035. , PA, USA (2007)
- [3] Awasthi, P., Sheffet, O.: Improved spectral-norm bounds for clustering. In: Gupta, A., Jansen, K., Rolim, J., Servedio, R. (eds.) Approximation, Randomization, and Combinatorial Optimization. Algorithms and Techniques. pp. 37–49. Springer Berlin Heidelberg, Berlin, Heidelberg (2012)
- [4] Dennis, D.K., Li, T., Smith, V.: Heterogeneity for the win: One-shot federated clustering. In: Meila, M., Zhang, T. (eds.) INTERNATIONAL CONFERENCE ON MACHINE LEARNING, VOL 139. Proceedings of Machine Learning Research, vol. 139 (2021), international Conference on Machine Learning (ICML), ELECTR NETWORK, JUL 18-24, 2021
- [5] Garst, S., de Reinders, M.: Federated k-means clustering. arXiv (Oct 2023), <http://arxiv.org/abs/2310.01195>
- [6] Grammenos, A., Mendoza-Smith, R., Mascolo, C., Crowcroft, J.A.: Federated pca with adaptive rank estimation. ArXiv **abs/1907.08059** (2019), <https://api.semanticscholar.org/CorpusID:197545218>
- [7] Holzer, P., Jacob, S.: Dynamically weighted federated k-means. arXiv (Nov 2023). <https://doi.org/10.48550/arXiv.2310.14858>, <http://arxiv.org/abs/2310.14858>
- [8] Hruschka, E., de Castro, L., Campello, R.: Evolutionary algorithms for clustering gene-expression data. In: Fourth IEEE International Conference on Data Mining (ICDM’04). pp. 403–403. Brighton, UK (01-04 novembre 2004). <https://doi.org/10.1109/ICDM.2004.10073>
- [9] Lloyd, S.: Least squares quantization in pcm. IEEE Transactions on Information Theory **28**(2), 129–137 (Mar 1982). <https://doi.org/10.1109/TIT.1982.1056489>
- [10] Mahalanobis, P.C.: On the generalised distance in statistics. Proceedings of the National Institute of Sciences of India **2**, 49–55 (1936)
- [11] McMahan, B., Moore, A., Ramage, D., Hampson, S., Arcas, B.A.y.: Communication-efficient learning of deep networks from decentralized data. In: Proceedings of the 20th International Conference on Artificial Intelligence and Statistics. vol. 54, pp. 1273–1282. PMLR (april 2017)
- [12] Md Tayeen, A.S., Misra, S., Cao, H., Harikumar, J.: Cafnet: Compressed autoencoder-based federated network for anomaly detection. In: MILCOM 2023 - 2023 IEEE Military Communications Conference (MILCOM). pp. 325–330 (2023). <https://doi.org/10.1109/MILCOM58377.2023.10356377>

- [13] Moustafa, N., Slay, J.: UNSW-NB15: a comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set). In: 2015 Military Communications and Information Systems Conference (MilCIS). IEEE (Nov 2015)
- [14] Nguyen, N.H., Nguyen, D.L., Nguyen, T.B., Nguyen, T.H., Pham, H.H., Nguyen, T.T., Le Nguyen, P.: Cadis: Handling cluster-skewed non-iid data in federated learning with clustered aggregation and knowledge distilled regularization. In: Altintas, I., Simmhan, Y., Varbanescu, A., Balaji, P., Prasad, A., Carnevale, L. (eds.) 2023 IEEE/ACM 23RD INTERNATIONAL SYMPOSIUM ON CLUSTER, CLOUD AND INTERNET COMPUTING, CCGRID. pp. 249+. IEEE; Assoc Comp Machinery; IEEE Comp Soc; IEEE Tech Comm Scalable Comp; IEEE Tech Community Cloud Comp; Assoc Comp Machinery, Special Interest Grp Comp Architecture; Meta; HPE; IBM; Google; Microsoft (2023). <https://doi.org/10.1109/CCGRID57682.2023.00032>, 23rd IEEE/ACM International Symposium on Cluster, Cloud and Internet Computing (CCGrid), Bangalore, INDIA, MAY 01-04, 2023
- [15] Nguyen, T.A., Le, L.T., Nguyen, T.D., Bao, W., Seneviratne, S., Hong, C.S., Tran, N.H.: Federated pca on grassmann manifold for iot anomaly detection. *IEEE/ACM Transactions on Networking* **32**(5), 4456–4471 (Oct 2024). <https://doi.org/10.1109/tnet.2024.3423780>, <http://dx.doi.org/10.1109/TNET.2024.3423780>
- [16] Prasad, M., Tripathi, S., Dahal, K.: Unsupervised feature selection and cluster center initialization based arbitrary shaped clusters for intrusion detection. *Computers & Security* **99**, 102062 (2020). <https://doi.org/https://doi.org/10.1016/j.cose.2020.102062>, <https://www.sciencedirect.com/science/article/pii/S0167404820303357>
- [17] Rousseeuw, P.J.: Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics* **20**, 53–65 (Nov 1987). [https://doi.org/10.1016/0377-0427\(87\)90125-7](https://doi.org/10.1016/0377-0427(87)90125-7), <https://www.sciencedirect.com/science/article/pii/0377042787901257>
- [18] Sharafaldin, I., Lashkari, A.H., Ghorbani, A.A.: Toward generating a new intrusion detection dataset and intrusion traffic characterization. In: 4th International Conference on Information Systems Security and Privacy (ICISSP). pp. 108–116. Portugal (01-04 novembre 2018). <https://doi.org/10.5220/0006639801080116>, <https://www.scitepress.org/papers/2018/66398/66398.pdf>
- [19] Sirisha, A., Chaitanya, K., Krishna, K., Kanumalli, S.: Intrusion detection models using supervised and unsupervised algorithms - a comparative estimation. *International Journal of Safety and Security Engineering* **11**, 51–58 (02 2021). <https://doi.org/10.18280/ijssse.110106>
- [20] Tensorflow: `tff.learning.algorithms.build_fed_kmeans` (2024), https://www.tensorflow.org/federated/api_docs/python/tff/learning/algorithms/build_fed_kmeans

- [21] World Trade Organization: 2023 was a big year for cybercrime – here’s how we can make our systems safer (2024), <https://www.weforum.org/agenda/2024/01/cybersecurity-cybercrime-system-safety/>
- [22] Yang, D., Hwang, M.: Unsupervised and ensemble-based anomaly detection method for network security. In: 2022 14th International Conference on Knowledge and Smart Technology (KST). pp. 75–79 (2022). <https://doi.org/10.1109/KST53302.2022.9729061>